Proyecto 2: Caja de una arepera

Tomas Santana

Octubre 2026

1 Descripción

Se debe desarrollar un programa en C++ que simule la caja de la arepera *Arepas Santa Rita* durante una jornada de trabajo: registra los pedidos de los clientes, aplica descuentos y recargos, y al cerrar la caja muestra un reporte de las ventas del día.

El proyecto se entrega **completo en una sola fecha**, pero está pensado para desarrollarse en dos partes:

Parte	**Qué se construye**	**Se puede hacer después de**
Parte 1	Un pedido completo: productos, cantidades, descuentos, método de pago y resumen.	Lab 4: `while`, `do-while`, `switch`, contadores, acumuladores, banderas y validación.
Parte 2	La jornada completa: menú, varios pedidos, funciones y reporte de cierre.	Lab 5: `for`, menús y funciones.

> **Recomendación:** terminar la Parte 1 antes de la clase del Lab 5.

2 Evaluación

La nota del proyecto tiene dos componentes: la implementación del programa y la explicación del mismo en clase.

$$\text{Nota final} = \text{Implementación} \times \text{Explicación}$$

Componente	**Escala**	**Qué se evalúa**
Implementación	0 a 20	Que el programa compile, cumpla todas las reglas y produzca los resultados correctos.
Explicación	0 a 1	Que el estudiante explique su código, prediga su comportamiento y lo modifique en clase.

> **Importante:** la explicación multiplica la nota. Un programa perfecto (20) que el estudiante no puede explicar (0) tiene una nota final de **0**. Un programa de 16 explicado con seguridad (1) tiene una nota final de **16**.

2.1 Implementación (0 a 20)

Parte	Criterio	Puntos
—	El programa compila sin errores con `g++` y se entrega a tiempo.	2
1	Registro de productos: valor centinela, códigos inválidos, validación de la cantidad y subtotal por línea (secciones 3.2 y 3.3).	4
1	Descuento por combo (sección 3.4).	2
1	Método de pago y resumen del pedido (secciones 3.5 y 3.6).	2
2	Menú principal y registro de varios pedidos (sección 4.1).	2
2	Funciones obligatorias, correctamente escritas y utilizadas (sección 4.2).	3
2	Totales de la jornada y pedido de mayor monto (sección 4.3).	2
2	Reporte: gráfico de barras, promedio por pedido y producto más vendido (sección 4.4).	2
—	Legibilidad: nombres descriptivos, indentación y comentarios.	1
	Total	**20**

El programa se probará con los casos de la sección 5 y con casos adicionales que no aparecen en este documento. Un programa que no compila tiene 0 en implementación.

2.2 Explicación (0 a 1)

Durante la presentación, con el programa abierto en VS Code, el profesor hará preguntas sobre el código. Las preguntas pueden consistir en:

- Explicar qué hace una parte específica del código y por qué se escribió de esa manera.
- Indicar qué mostrará el programa con ciertos datos antes de ejecutarlo.
- Realizar en el momento un cambio pequeño en las reglas y demostrar que funciona (por ejemplo, agregar un producto o cambiar un porcentaje).

Valor	Criterio
1	Explica cualquier parte con seguridad, predice la salida correctamente y realiza el cambio solicitado.
0.75	Explica casi todo; duda en algún detalle o necesita una pista pequeña para realizar el cambio.
0.5	Explica la idea general, pero no justifica sus decisiones ni logra realizar el cambio.
0.25	Reconoce partes del código, pero no explica cómo funciona.
0	No puede explicar su propio código.

> Cualquier ayuda recibida (compañeros, internet, herramientas de IA) debe ser declarada, según lo establecido en el sílabo. En todos los casos, el estudiante debe comprender cada línea de su programa: la presentación es individual.

3 Parte 1: Un pedido

3.1 Carta de productos

Código	Producto	Precio	Tipo
1	Arepa reina pepiada	$4.00	Arepa
2	Arepa pabellón	$4.50	Arepa
3	Empanada	$1.50	—
4	Jugo natural	$2.00	Bebida
5	Malta	$1.50	Bebida

3.2 Registro de productos

- El programa pide el código de un producto. El código `0` indica que el pedido terminó (valor centinela).
- Si el código no es 0 ni un código de la carta, se muestra `Codigo invalido` y se vuelve a pedir un código.
- Por cada producto válido se pide la cantidad y se muestra el subtotal de esa línea (precio × cantidad).
- Un mismo producto puede pedirse más de una vez en el mismo pedido.
- Si el cliente marca `0` sin haber pedido ningún producto, se muestra `Pedido vacio` y no se realiza ningún otro cálculo.

3.3 Validación de la cantidad

- La cantidad de cada producto debe estar entre 1 y 10.
- Si no lo está, se muestra `Cantidad invalida` y se vuelve a pedir **solo la cantidad**, hasta que sea válida.

3.4 Descuento por combo

- Subtotal = suma de los subtotales de todas las líneas del pedido.
- Si el pedido tiene **al menos una arepa y al menos una bebida**, se descuenta el 10% del subtotal.
- Total = subtotal − descuento por combo.

Pista: para el combo, usar dos variables `bool` (banderas) que empiecen en `false` y cambien a `true` cuando se pida una arepa o una bebida.

3.5 Método de pago

Después de terminar el pedido se pide el método de pago:

Opción	Método	Regla
1	Tarjeta	Se aplica un recargo del 3% sobre el total (después de los descuentos).
2	Pago móvil	Sin recargo. Además del total en dólares, se muestra el equivalente en bolívares con una tasa fija de **900 Bs por dólar**.

Si la opción no es 1 ni 2, se muestra `Opcion invalida` y se vuelve a pedir.

3.6 Resumen del pedido

Al final se muestra: cantidad de artículos, subtotal, descuento por combo, recargo, total y, si el pago fue por pago móvil, el total en bolívares.

Todos los montos se muestran con dos decimales. Para lograrlo, agregar `#include <iomanip>` al inicio del archivo y escribir esta línea al comienzo de `main`:

```
std::cout << std::fixed << std::setprecision(2);
```

3.7 Ejemplo de ejecución

```
Codigo (0 para terminar): 2
Cantidad: 3
  Arepa pabellon x3 = $13.50
Codigo (0 para terminar): 5
Cantidad: 12
Cantidad invalida
Cantidad: 3
  Malta x3 = $4.50
Codigo (0 para terminar): 0
Metodo de pago (1 = tarjeta, 2 = pago movil): 2
-------------------------
Articulos:       6
Subtotal:        $18.00
Desc. combo:     $1.80
Recargo:         $0.00
Total:           $16.20
Total en Bs:     14580.00
-------------------------
```

El formato puede variar, pero la salida debe incluir todos estos valores.

4 Parte 2: La jornada completa

4.1 Menú principal

El programa muestra un menú que se repite hasta que se elige cerrar la caja:

```
===== AREPAS SANTA RITA =====
1. Nuevo pedido
2. Ver carta
3. Reporte parcial
4. Cerrar caja y salir
Opcion:
```

- **Nuevo pedido:** registra un pedido completo siguiendo todas las reglas de la Parte 1. Los pedidos vacíos no se cuentan.
- **Ver carta:** muestra la tabla de productos con su código y precio.
- **Reporte parcial:** muestra el reporte (sección 4.4) con los pedidos registrados hasta el momento.
- **Cerrar caja y salir:** muestra el reporte final y termina el programa.
- Una opción inválida muestra `Opcion invalida` y se vuelve a pedir.

4.2 Funciones obligatorias

El programa debe definir y usar, como mínimo, las siguientes funciones:

Función	Qué hace
`double precioProducto(int codigo)`	Devuelve el precio del producto usando un `switch`.
`int leerEnteroEnRango(int min, int max)`	Lee un número entero y lo vuelve a pedir hasta que esté entre `min` y `max`. Debe usarse al menos para la opción del menú y el método de pago.
`void mostrarCarta()`	Muestra la carta de productos.
`void mostrarLinea(int largo)`	Muestra una línea separadora de `largo` guiones, usando `for`.
`void mostrarBarra(int cantidad)`	Muestra `cantidad` asteriscos seguidos, usando `for`.

Se pueden agregar otras funciones si ayudan a organizar el programa.

4.3 Totales de la jornada

Durante la jornada se deben acumular:

- La cantidad de pedidos registrados (sin contar los vacíos).
- Las unidades vendidas de cada producto.
- El total recaudado con tarjeta y el total recaudado con pago móvil, en dólares (incluyendo el recargo de tarjeta).
- El monto del pedido más alto de la jornada.

4.4 Reporte

El reporte (parcial o final) muestra los totales anteriores, el total recaudado (tarjeta + pago móvil) y un gráfico de barras con las unidades vendidas de cada producto, dibujado con `mostrarBarra`:

```
----- REPORTE -----
Pedidos:              3
Recaudado tarjeta:    $7.21
Recaudado pago movil: $25.20
Total recaudado:      $32.41
Pedido mas alto:      $16.20

Reina pepiada  | *** (3)
Pabellon       | *** (3)
Empanada       | ** (2)
Jugo natural   | * (1)
Malta          | *** (3)
```

Al cerrar la caja, el reporte final muestra además:

- El **promedio por pedido** (total recaudado ÷ cantidad de pedidos). Si no hubo pedidos, se muestra `No hubo pedidos` en lugar del promedio.
- El **producto más vendido** (el de más unidades). Si hay empate, se muestra el de menor código.

```
----- CIERRE -----
Pedidos:              3
Recaudado tarjeta:    $7.21
Recaudado pago movil: $25.20
Total recaudado:      $32.41
Pedido mas alto:      $16.20
Promedio por pedido:  $10.80

Reina pepiada  | *** (3)
```

```
Pabellon       | *** (3)
Empanada       | ** (2)
Jugo natural   | * (1)
Malta          | *** (3)

Producto más vendido: Reina Pepiada.
```

5 Casos de prueba

El programa debe probarse con todos los casos siguientes antes de entregarlo. Superar estos casos no garantiza la nota completa de implementación, ya que el programa también se evaluará con otros casos.

Junto con este enunciado se entrega el archivo `pruebas.cpp`, que ejecuta automáticamente todos los casos de esta sección. Para usarlo:

1. Colocar `pruebas.cpp` en la misma carpeta que `proyecto2.cpp`.
2. Abrir una terminal en esa carpeta y ejecutar `init.bat`.
3. Compilar y ejecutar el probador:

```
g++ pruebas.cpp -o pruebas.exe
.\pruebas.exe
```

El probador compila `proyecto2.cpp`, lo ejecuta con cada caso de prueba e indica si el caso `PASA` o `FALLA`. Para cada caso que falla, muestra los datos de entrada, qué no se encontró y la salida completa del programa. Como cada caso empieza eligiendo una opción del menú principal, el probador funciona con el programa completo (Parte 2).

Para que el probador reconozca correctamente la salida del programa, se deben tener en cuenta las siguientes condiciones:

- Todos los montos deben mostrarse con dos decimales (sección 3.6). El probador solo reconoce números con decimales, como `9.27` o `0.00`.
- En el resumen de un pedido, los montos deben mostrarse en el orden del ejemplo de ejecución: subtotal, descuento por combo, recargo, total y, si corresponde, total en bolívares.
- En el cierre de caja, los montos deben mostrarse en el orden del ejemplo: recaudado con tarjeta, recaudado con pago móvil, total recaudado, pedido más alto y promedio por pedido.
- Los mensajes deben escribirse igual que en el enunciado, sin acentos (por ejemplo, `Codigo invalido`). No se distingue entre mayúsculas y minúsculas.

5.1 Un pedido

Cada producto se indica como `código×cantidad`. Pago: 1 = tarjeta, 2 = pago móvil.

#	Productos	Pago	Art.	Subt.	Combo	Rec.	Total	Total Bs
1	`1×2`, `4×1`	1	3	10.00	1.00	0.27	9.27	—
2	`3×4`	2	4	6.00	0.00	0.00	6.00	5400.00
3	`2×3`, `5×3`	2	6	18.00	1.80	0.00	16.20	14580.00
4	`1×1`, `3×2`	1	3	7.00	0.00	0.21	7.21	—
5	`4×2`, `5×1`	2	3	5.50	0.00	0.00	5.50	4950.00
6	`3×5`, `3×5`	1	10	15.00	0.00	0.45	15.45	—

5.2 Validaciones

#	Entradas	Resultado esperado
7	Código `7`, luego `1×1`, `0`, pago `2`	`Codigo invalido`; total $4.00, 3600.00 Bs.
8	Código `1` con cantidad `0`, luego `11`, luego `2`; `4×1`, `0`, pago `2`	Dos veces `Cantidad invalida`; total $9.00, 8100.00 Bs.
9	`1×1`, `4×1`, `0`, pago `3`, luego `1`	`Opcion invalida`; total $5.56.
10	Código `0` inmediatamente	`Pedido vacio`.

5.3 Jornada completa

#	Secuencia	Resultado esperado al cerrar caja
11	Pedido `1×2`, `4×1` con pago `2`; pedido `2×3`, `5×3` con pago `2`; pedido vacío; pedido `1×1`, `3×2` con pago `1`; cerrar caja.	3 pedidos. Tarjeta $7.21, pago móvil $25.20, total $32.41. Pedido más alto $16.20. Barras: 3, 3, 2, 1, 3. Promedio $10.80. Más vendido: Arepa reina pepiada (empate en 3 unidades; gana el menor código).
12	Cerrar caja sin registrar pedidos.	0 pedidos, todos los totales en $0.00, `No hubo pedidos`.

6 Restricciones

- Usar únicamente los contenidos de los laboratorios 1 a 5: entrada y salida, variables (`int`, `double`, `bool`, `char`), operadores, condicionales, `while`, `do-while`, `for`, `switch` y funciones.
- No usar arreglos ni `string`.
- Todo el programa debe estar en un único archivo llamado `proyecto2.cpp`.

7 Entrega

1. Entregar el archivo `proyecto2.cpp` en la fecha y por el medio indicados.
2. Si el estudiante no realiza la presentación, la explicación tiene un valor de 0 y, por lo tanto, la nota final es 0, salvo uso del comodín o retraso justificado según lo establecido en el sílabo.